

Aula de Reforço — FOREIGN KEY + JOIN

1) Entendendo o Problema

Quando tu tem várias tabelas no banco, tu precisa “ligar” elas.
Do contrário, os dados ficam soltos e não fazem sentido.

Exemplo do mundo real:

Tabela	O que guarda
cliente	informações da pessoa
pedido	quando e quem comprou
produto	o que está sendo vendido
item_pedido	quais produtos tem em cada pedido

Chave estrangeira (FOREIGN KEY) é o *elo* entre essas tabelas.

2) Chave Primária (PK) e Chave Estrangeira (FK)

PK (PRIMARY KEY)

É a *identidade do registro*.

Não pode repetir e não pode ser nula.

```
id INT AUTO_INCREMENT PRIMARY KEY
```

FK (FOREIGN KEY)

É uma coluna que aponta para a **PK de outra tabela**.

```
FOREIGN KEY(id_cliente) REFERENCES cliente(id)
```

Tradução mental:

“id_cliente deve existir na tabela cliente”.

3) Criação das Tabelas com FK

```
CREATE TABLE cliente (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100)  
);  
  
CREATE TABLE pedido (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  id_cliente INT,  
  data_pedido DATE,  
  FOREIGN KEY(id_cliente) REFERENCES cliente(id)  
);
```

Interpretação:

- Cada **pedido** pertence a **um cliente**.
- Não tem como criar pedido para cliente inexistente.

4) Por que Usar FK?

- Evita bagunça (integridade referencial).
- Impede "dados órfãos" (pedidos sem cliente).
- Mantém o banco coerente.

Quando a FK existe, o banco **protege seus dados por ti**.

5) Agora o Bicho Papão: JOIN

JOIN serve para **buscar dados em duas ou mais tabelas ao mesmo tempo**.

5.1 INNER JOIN

Traz **apenas** registros que existem nas duas tabelas.

```
SELECT cliente.nome, pedido.data_pedido  
FROM cliente  
JOIN pedido ON cliente.id = pedido.id_cliente;
```

Tradução:

“Me mostra o nome do cliente e a data do pedido,
conectando cliente.id com pedido.id_cliente.”

5.2 LEFT JOIN

Traz **todos** os registros da tabela da esquerda, mesmo que não tenha correspondência na tabela da direita.

```
SELECT cliente.nome, pedido.data_pedido  
FROM cliente  
LEFT JOIN pedido ON cliente.id = pedido.id_cliente;
```

Usa quando:

“Quero ver os clientes, mesmo os que nunca compraram.”

5.3 RIGHT JOIN (menos usado)

Inverso do LEFT.

5.4 JOIN entre 4 tabelas

```
SELECT c.nome AS cliente,  
       p.id AS pedido,  
       pr.nome AS produto,  
       i.quantidade  
FROM cliente c  
JOIN pedido p ON c.id = p.id_cliente  
JOIN item_pedido i ON p.id = i.id_pedido  
JOIN produto pr ON pr.id = i.id_produto;
```

Tradução em voz alta (pra decorar):

cliente → pedido → item_pedido → produto

Se tu contar essa história, tu nunca mais erra JOIN.

6) Exercícios (Pra Fixar)

1) Liste o nome do cliente e o número do pedido

```
SELECT c.nome, p.id  
FROM cliente c
```

```
JOIN pedido p ON c.id = p.id_cliente;
```

2) Liste produto e quantidade vendida

```
SELECT pr.nome, i.quantidade  
FROM produto pr  
JOIN item_pedido i ON pr.id = i.id_produto;
```

3) Liste o total gasto por pedido

```
SELECT p.id,  
       SUM(pr.preco * i.quantidade) AS total  
FROM pedido p  
JOIN item_pedido i ON p.id = i.id_pedido  
JOIN produto pr ON pr.id = i.id_produto  
GROUP BY p.id;
```

7) Macete pra Nunca Mais Travar

Parte	O que faz	Pergunta que tu te faz
SELECT	O que quero ver	“Qual informação eu quero?”
FROM	Tabela base	“Onde começa a história?”
JOIN	Caminho	“Quem está ligado a quem?”
ON	Condição de ligação	“Quais campos se conectam?”

JOIN é contar história.

Se tu sabe a história, tu escreve a query.